

Практичне заняття №8

Модуль замовлень: фінальний проєкт MiniShop

Дисципліна: Високорівневі мови програмування та фреймворки

Це заняття відрізняється від попередніх. Тут ви отримуєте технічне завдання, а не покроковий туторіал. Ви вже маєте всі знання — тепер час їх застосувати.

Мета

Самостійно створити OrdersModule для MiniShop — від Entity до Swagger — використовуючи ВСЕ, що було вивчено за курс: Entity + міграції, DTO + class-validator, JWT Guards + RBAC, Interceptors, Exception Filters, Swagger, Redis кешування, пагінацію. Вперше реалізувати транзакційну бізнес-логіку (перевірка stock, підрахунок totalPrice) та ownership check (користувач бачить тільки свої замовлення).

Як працювати з цією практичною

На попередніх практичних ви отримували готовий код для кожного кроку. Тут — ні. Замість цього ви отримуєте:

- **Вимоги** — що повинен робити ендпоінт, які поля в Entity, який формат відповіді
- **Підказки** — для нових концепцій (транзакції, nested validation, ownership) — готовий код
- **Чеклісти** — для перевірки, що все працює

Все інше — ваші рішення. Назви методів, структура сервісу, порядок перевірок — це ваша відповідальність. Саме так виглядає реальна робота розробника: ви отримуєте задачу, а не інструкцію.

Важливо: Якщо застрягли — перегляньте як аналогічна задача вирішена в ProductsModule або CategoriesModule. Паттерни однакові, тільки Entity та бізнес-логіка відрізняються.

Передумови

Виконані Практичні №1–7: MiniShop API з модулями Auth, Users, Categories, Products. JWT-автентифікація, RBAC, Swagger, Redis кешування, пагінація. Seed-скрипт наповнив базу тестовими даними (30 продуктів, 3 категорії).

Що здаємо

Лінк на GitHub репозиторій, в якому є:

- README.md з фінальним звітом: повна структура проєкту, таблиця BCIX ендпоінтів, вивід тестів
- Order + OrderItem Entity з міграцією
- OrdersModule: controller, service, DTOs з повною валідацією
- Транзакційне створення замовлення з перевіркою stock
- Ownership check: user бачить тільки свої замовлення
- Пагінація + фільтрація за статусом
- Swagger-документація для всіх нових ендпоінтів
- Мінімум 4 нових коміти

Вправа 1 — Order + OrderItem Entity та міграція

Вимоги

Створити дві нових Entity: Order та OrderItem. Згенерувати та запустити міграцію.

Order Entity — таблиця orders:

Поле	Тип	Обмеження	Коментар
id	number	PrimaryGeneratedColumn	Auto-increment
status	enum	default: 'pending'	OrderStatus enum
totalPrice	decimal(10,2)	default: 0	Підраховується при створенні
user	ManyToOne → User	eager: false	Хто замовив
items	OneToMany → OrderItem	eager: true, cascade: true	Позиції замовлення
createdAt	Date	CreateDateColumn	

OrderItem Entity — таблиця order_items:

Поле	Тип	Обмеження	Коментар
------	-----	-----------	----------

id	number	PrimaryGeneratedColumn	
quantity	int	не менше 1	Кількість одиниць
price	decimal(10,2)		Ціна на момент замовлення (snapshot)
order	ManyToOne → Order	onDelete: CASCADE	До якого замовлення належить
product	ManyToOne → Product	eager: true	Який продукт замовлено

OrderStatus enum:

```
export enum OrderStatus {
  PENDING = 'pending',
  CONFIRMED = 'confirmed',
  SHIPPED = 'shipped',
  DELIVERED = 'delivered',
  CANCELLED = 'cancelled',
}
```

💡 *Поле price в OrderItem — це snapshot ціни на момент замовлення. Навіть якщо завтра ціна продукту зміниться — замовлення зберігає стару ціну. Це стандартна практика в e-commerce.*

Ваші кроки

- 1) Створити enum src/common/enums/order-status.enum.ts
- 2) Створити src/orders/entities/order.entity.ts та order-item.entity.ts
- 3) Зареєструвати Entity в app.module.ts (масив entities в TypeOrmModule)
- 4) Згенерувати міграцію та перезапустити

```
docker compose run --rm app npx ts-node \
  -r tsconfig-paths/register \
  ./node_modules/typeorm/cli.js migration:generate \
  src/migrations/CreateOrders -d src/data-source.ts
```

```
docker compose up --build -d
```

- 5) Перевірити що таблиці створені:

```
docker compose exec postgres psql -U nestuser -d nestdb \
  -c "\d orders"
docker compose exec postgres psql -U nestuser -d nestdb \
  -c "\d order_items"
```

Чекліст

- Таблиця orders має колонки: id, status, totalPrice, userId, createdAt
- Таблиця order_items має колонки: id, quantity, price, orderId, productId
- Foreign keys створені коректно
- Міграція в папці src/migrations/

Вправа 2 — DTO з вкладеною валідацією (нова концепція)

Вимоги

Створити CreateOrderDto, який приймає масив items — кожен з productId та quantity. Це перший раз, коли ви валідуєте вкладені об'єкти.

Формат запиту POST /api/orders:

```
{
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 5, "quantity": 1 }
  ]
}
```

Підказка: @ValidateNested (нова концепція)

class-validator вміє валідувати вкладені об'єкти через @ValidateNested. Але йому потрібна допомога від @Type, щоб знати, в який клас перетворити plain object:

```
// create-order-item.dto.ts
import { IsInt, Min } from 'class-validator';
import { ApiProperty } from '@nestjs/swagger';

export class CreateOrderItemDto {
  @ApiProperty({ example: 1, description: 'ID продукту' })
  @IsInt()
  productId: number;
```

```

    @ApiProperty({ example: 2, description: 'Кількість',
        minimum: 1 })
    @IsInt()
    @Min(1)
    quantity: number;
}

```

```

// create-order.dto.ts
import {
    isArray,
    ValidateNested,
    ArrayMinSize,
} from 'class-validator';
import { Type } from 'class-transformer';
import { ApiProperty } from '@nestjs/swagger';
import { CreateOrderItemDto }
    from './create-order-item.dto';

export class CreateOrderDto {
    @ApiProperty({
        type: [CreateOrderItemDto],
        description: 'Масив позицій замовлення',
        example: [
            { productId: 1, quantity: 2 },
            { productId: 5, quantity: 1 },
        ],
    })
    @IsArray()
    @ArrayMinSize(1)
    @ValidateNested({ each: true })
    @Type(() => CreateOrderItemDto)
    items: CreateOrderItemDto[];
}

```

Декоратор	Що робить
@ValidateNested({ each: true })	Валідує кожен елемент масиву як окремий DTO-об'єкт
@Type(() => CreateOrderItemDto)	Каже class-transformer перетворити plain object у CreateOrderItemDto. Без цього декоратори валідації на вкладеному об'єкті не спрацюють
@ArrayMinSize(1)	Замовлення не може бути порожнім — мінімум 1 позиція

Додатково створіть самостійно

- UpdateOrderStatusDto — з полем status (@IsEnum(OrderStatus))
- OrderQueryDto — page, pageSize, status (фільтр, @IsOptional @IsEnum)

Чекліст

- POST з порожнім items: [] → 400 (ArrayMinSize)
- POST з { productId: 'abc' } → 400 (IsInt)
- POST з { productId: 1, quantity: 0 } → 400 (Min(1))
- POST з валідними даними → проходить валідацію

Вправа 3 — OrdersModule, Controller, Service (самостійно)

Вимоги до ендпоінтів

Method	URL	Auth	Role	Опис
POST	/api/orders	JWT	user, admin	Створити замовлення
GET	/api/orders	JWT	user, admin	Мої замовлення (user) / Всі (admin)
GET	/api/orders/:id	JWT	user, admin	Одне замовлення (ownership check)
PATCH	/api/orders/:id/status	JWT	admin	Змінити статус
DELETE	/api/orders/:id	JWT	admin	Видалити замовлення

Ваші кроки

- 1) Створити OrdersModule (module, controller, service) — аналогічно до ProductsModule
- 2) Імпортувати TypeOrmModule.forFeature([Order, OrderItem]), AuthModule (для Guard), ProductsModule або TypeOrmModule.forFeature([Product]) (для перевірки stock)

3) Зареєструвати OrdersModule в AppModule

4) Реалізувати 5 ендпоінтів з Guards та Swagger-декораторами

Захист ендпоінтів:

- Всі ендпоінти потребують @UseGuards(JwtAuthGuard) — замовлення не можуть бути анонімними
- POST /api/orders — доступний будь-якому залогіненому (user або admin)
- PATCH /api/orders/:id/status та DELETE — тільки @Roles(Role.ADMIN)
- GET ендпоінти — залогінений, але з ownership check у сервісі (не в Guard)

💡 Не забудьте @CurrentUser() для отримання поточного користувача. POST /api/orders потребує userId для збереження замовлення. GET /api/orders потребує userId для фільтрації.

Чекліст

- Swagger показує секцію Orders з 5 ендпоінтами
- POST /api/orders без токена → 401
- Маршрути з ParseIntPipe для :id

Вправа 4 — Бізнес-логіка створення замовлення (нова концепція)

Вимоги

Метод create в OrdersService повинен:

- 1) Знайти кожен продукт за productId з масиву items
- 2) Для кожного: перевірити що product.stock >= item.quantity
- 3) Якщо stock недостатній — повернути 400 Bad Request з повідомленням який саме продукт
- 4) Зменшити stock кожного продукту на quantity
- 5) Підрахувати totalPrice = сума (product.price × item.quantity) по всіх items
- 6) Зберегти Order з OrderItems та оновленими продуктами
- 7) Все це — в одній транзакції: якщо щось впаде, stock не зміниться

Підказка: TypeORM транзакції через QueryRunner (нова концепція)

Транзакція гарантує атомарність: або ВСІ зміни зберігаються, або ЖОДНА.
Якщо save order впав — stock продуктів залишиться попереднім.

```
import { DataSource } from 'typeorm';

@Injectable()
export class OrdersService {
  constructor(
    @InjectRepository(Order)
    private readonly orderRepo: Repository<Order>,
    @InjectRepository(Product)
    private readonly productRepo: Repository<Product>,
    private readonly dataSource: DataSource,
    @Inject(CACHE_MANAGER)
    private readonly cacheManager: Cache,
  ) {}

  async create(
    dto: CreateOrderDto,
    userId: number,
  ): Promise<Order> {
    // Створюємо QueryRunner для транзакції
    const qr = this.dataSource.createQueryRunner();
    await qr.connect();
    await qr.startTransaction();

    try {
      let totalPrice = 0;
      const orderItems: OrderItem[] = [];

      for (const item of dto.items) {
        // Знайти продукт
        const product = await qr.manager
          .findOne(Product, {
            where: { id: item.productId },
          });

        if (!product) {
          throw new NotFoundException(
            `Product #${item.productId} not found`,
          );
        }

        // Перевірити stock
```



```
if (product.stock < item.quantity) {
  throw new BadRequestException(
    `Insufficient stock for "${product.name}":` +
    ` available ${product.stock},` +
    ` requested ${item.quantity}`,
  );
}

// Зменшити stock
product.stock -= item.quantity;
await qr.manager.save(product);

// Створити OrderItem (snapshot ціни)
const orderItem = qr.manager.create(
  OrderItem,
  {
    product,
    quantity: item.quantity,
    price: product.price,
  },
);
orderItems.push(orderItem);

totalPrice +=
  Number(product.price) * item.quantity;
}

// Створити і зберегти Order
const order = qr.manager.create(Order, {
  user: { id: userId } as any,
  items: orderItems,
  totalPrice,
  status: OrderStatus.PENDING,
});

const saved = await qr.manager.save(order);

// Все ОК – зберігаємо транзакцію
await qr.commitTransaction();

// Інвалідувати кеш продуктів
// (stock змінився!)
await this.clearProductsCache();

return saved;
```

```

    } catch (error) {
      // Щось пішло не так — відкат
      await qr.rollbackTransaction();
      throw error;
    } finally {
      // Завжди звільнити з'єднання
      await qr.release();
    }
  }

  private async clearProductsCache() {
    const keys: string[] = await this.cacheManager
      .store.keys('products:*');
    if (keys.length > 0) {
      await Promise.all(
        keys.map((k) => this.cacheManager.del(k)),
      );
    }
  }
}

```

Крок	Що відбувається
createQueryRunner()	Створює окреме з'єднання з БД для транзакції
startTransaction()	Починає транзакцію — всі зміни тимчасові
qr.manager.save()	Зберігає через транзакційний менеджер (не через this.productRepo!)
commitTransaction()	Фіксує всі зміни — тепер вони у базі
rollbackTransaction()	Відкочує все — ніби нічого не було
release()	Звільняє з'єднання — обов'язково, навіть при помилці (finally)

⚠ Всередині транзакції використовуйте qr.manager замість this.productRepo. Якщо зберігати через repo — зміни будуть поза транзакцією і rollback їх не скасує.

Чекліст

- POST з існуючими продуктами → 201, stock зменшився
- POST з неіснуючим productId → 404
- POST з quantity > stock → 400 з назвою продукту

- При помилці stock НЕ зменшується (rollback)
- Кеш продуктів інвалідовано після створення замовлення

Вправа 5 — Ownership check (нова концепція)

Вимоги

GET /api/orders — user бачить тільки свої замовлення, admin — всі. GET /api/orders/:id — user може отримати тільки СВОЄ замовлення. Спроба отримати чуже → 403 Forbidden. Це захист від IDOR-атаки (Insecure Direct Object Reference) з Лекції 7.

Підказка: ownership check у сервісі

```
async findAll(
  query: OrderQueryDto,
  userId: number,
  userRole: Role,
) {
  const qb = this.orderRepo
    .createQueryBuilder('order')
    .leftJoinAndSelect('order.items', 'item')
    .leftJoinAndSelect('item.product', 'product');

  // Ownership: user бачить тільки свої
  if (userRole !== Role.ADMIN) {
    qb.andWhere('order.userId = :userId',
      { userId });
  }

  // Фільтр за статусом
  if (query.status) {
    qb.andWhere('order.status = :status',
      { status: query.status });
  }

  // Пагінація + сортування
  // ... аналогічно до ProductsService ...
}

async findOne(
  id: number,
  userId: number,
```

```

        userRole: Role,
    ): Promise<Order> {
        const order = await this.orderRepo.findOne({
            where: { id },
            relations: ['items', 'items.product', 'user'],
        });

        if (!order) {
            throw new NotFoundException(
                `Order #${id} not found`,
            );
        }

        // Ownership check
        if (
            userRole !== Role.ADMIN &&
            order.user.id !== userId
        ) {
            throw new ForbiddenException(
                'You can only view your own orders',
            );
        }

        return order;
    }

```

В контролері:

```

@Get()
@UseGuards(JwtAuthGuard)
@ApiOperation({
    summary: 'Мої замовлення (user) / Всі (admin)',
})
findAll(
    @Query() query: OrderQueryDto,
    @CurrentUser('sub') userId: number,
    @CurrentUser('role') role: Role,
) {
    return this.ordersService.findAll(
        query, userId, role,
    );
}

```

💡 *Ownership check відбувається в СЕРВІСІ, а не в Guard. Чому? Guard перевіряє ролі — загальні правила. Ownership — це бізнес-логіка конкретного ресурсу. Guard не знає, кому належить замовлення — це знає тільки сервіс.*

Чекліст

- GET /api/orders з токеном user — тільки його замовлення
- GET /api/orders з токеном admin — всі замовлення
- GET /api/orders/1 з токеном user-власника → 200
- GET /api/orders/1 з токеном іншого user → 403
- GET /api/orders/1 з токеном admin → 200 (навіть чуже)

Вправа 6 — Зміна статусу замовлення (самостійно)

Вимоги

PATCH /api/orders/:id/status — тільки admin. Приймає { status: 'confirmed' }.
Валідація переходів: не всі переходи дозволені.

Дозволені переходи статусів:

Поточний статус	Дозволені переходи
pending	confirmed, cancelled
confirmed	shipped, cancelled
shipped	delivered
delivered	(фінальний — зміна заборонена)
cancelled	(фінальний — зміна заборонена)

Ваші кроки

- 1) Створити UpdateOrderStatusDto з @IsEnum(OrderStatus) та Swagger-декоратором
- 2) Реалізувати метод updateStatus(id, dto) в OrdersService
- 3) Перевірити що поточний статус дозволяє перехід. Якщо ні — 400 Bad Request з описом
- 4) При cancelled — бонусне завдання: повернути stock продуктів назад

💡 Повернення stock при cancelled — це не обов'язково, але дуже рекомендовано. Це ще одна транзакція: знайти OrderItems → для кожного збільшити product.stock += quantity → зберегти. Спробуйте реалізувати самостійно.

Чекліст

- pending → confirmed → 200
- pending → shipped → 400 (неможливий перехід)
- delivered → будь-що → 400 (фінальний статус)
- PATCH без admin-токена → 403

Вправа 7 — Повне тестування: curl-сценарій

Опис

Прогнати повний end-to-end сценарій від реєстрації до перевірки ownership. Цей сценарій — ваш фінальний інтеграційний тест MiniShop.

Сценарій

1. Підготовка — переконатись що seed-дані є:

```
docker compose run --rm app npm run seed
curl "http://localhost:3000/api/products?pageSize=5"
# Перевірити що є продукти з stock > 0
```

2. Зареєструвати двох користувачів:

```
# User A
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@test.com","password":"pass12345","name":"Alice"}'

# User B
curl -X POST http://localhost:3000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"bob@test.com","password":"pass12345","name":"Bob"}'
```

3. Залогінути обох + отримати admin-токен:

```
# Login Alice
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@test.com","password":"pass12345"}'
# Зберегти ALICE_TOKEN

# Login Bob
# Зберегти BOB_TOKEN

# Login Admin (створений раніше)
```

```
# Зберегти ADMIN_TOKEN
```

4. Alice створює замовлення:

```
curl -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ALICE_TOKEN" \
  -d '{"items":[{"productId":1,"quantity":2},{"productId":5,"quantity":1}]}'
# Очікується: 201, totalPrice = (price1 * 2) + (price5 * 1)
# Зберегти ORDER_ID з відповіді
```

5. Перевірити що stock зменшився:

```
curl http://localhost:3000/api/products/1
# stock має бути на 2 менше ніж до замовлення
```

6. Alice бачить своє замовлення:

```
curl http://localhost:3000/api/orders \
  -H "Authorization: Bearer $ALICE_TOKEN"
# Очікується: 1 замовлення
```

7. Bob НЕ бачить замовлення Alice:

```
curl http://localhost:3000/api/orders/$ORDER_ID \
  -H "Authorization: Bearer $BOB_TOKEN"
# Очікується: 403 Forbidden
```

8. Admin бачить всі замовлення:

```
curl http://localhost:3000/api/orders \
  -H "Authorization: Bearer $ADMIN_TOKEN"
# Очікується: всі замовлення від усіх користувачів
```

9. Admin змінює статус:

```
curl -X PATCH http://localhost:3000/api/orders/$ORDER_ID/status \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"status": "confirmed"}'
# Очікується: 200, status = confirmed
```

10. Невалідний перехід статусу:

```
curl -X PATCH http://localhost:3000/api/orders/$ORDER_ID/status \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
```

```
-d '{"status": "pending"}'
```

Очікується: 400 (confirmed → pending неможливий)

11. Недостатній stock:

```
# Спробувати замовити більше ніж є на складі
curl -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $BOB_TOKEN" \
  -d '{"items":[{"productId":1,"quantity":99999}]}'
```

Очікується: 400, Insufficient stock for ...

12. Swagger — відкрити <http://localhost:3000/api/docs>:

Перевірити що секція Orders показує всі 5 ендпоінтів з описами, прикладами та можливими відповідями.

Вправа 8 — Фінальний README та коміти

Інструкції

1) Зробити коміти (мінімум 4):

```
git add src/common/enums/order-status.enum.ts
git add src/orders/entities/ src/migrations/
git commit -m "add Order, OrderItem entities with migration"
```

```
git add src/orders/dto/
git commit -m "add order DTOs with nested validation"
```

```
git add src/orders/orders.module.ts
git add src/orders/orders.service.ts
git add src/orders/orders.controller.ts
git commit -m "add OrdersModule with transactional create, ownership check"
```

```
git add src/app.module.ts README.md
git commit -m "register OrdersModule, final readme"
```

```
git push
```

2) Оновити README.md — фінальна версія:

```
## Student
- Name: <ПІБ>
- Group: <Група>
```


MiniShop API — Фінальний проект

REST API інтернет-магазину на NestJS + PostgreSQL + Redis.

Технології

- NestJS + TypeScript
- PostgreSQL + TypeORM (міграції, QueryBuilder)
- Redis (кешування з інвалідацією)
- JWT автентифікація + RBAC авторизація
- class-validator + class-transformer
- Swagger / OpenAPI

Запуск

```
```bash
cp .env.example .env
docker compose up --build
docker compose run --rm app npm run seed
```
```

Swagger UI

<http://localhost:3000/api/docs>

API Endpoints

Auth

| Method | URL | Auth | Опис |
|--------|----------------|------|-------------|
| POST | /auth/register | - | Реєстрація |
| POST | /auth/login | - | Логін → JWT |

Categories

| Method | URL | Auth | Опис |
|--------|---------------------|-------|----------|
| GET | /api/categories | - | Список |
| GET | /api/categories/:id | - | Одна |
| POST | /api/categories | admin | Створити |
| PATCH | /api/categories/:id | admin | Оновити |
| DELETE | /api/categories/:id | admin | Видалити |

Products

| Method | URL | Auth | Опис |
|--------|-------------------|------|------------------------------|
| GET | /api/products | - | Список + pagination + filter |
| GET | /api/products/:id | - | Один |

| | | | |
|--------|-------------------|-------|----------|
| POST | /api/products | admin | Створити |
| PATCH | /api/products/:id | admin | Оновити |
| DELETE | /api/products/:id | admin | Видалити |

Orders

| Method | URL | Auth | Опис |
|--------|------------------------|-------|---------------------|
| POST | /api/orders | user | Створити замовлення |
| GET | /api/orders | user | Мої / Всі (admin) |
| GET | /api/orders/:id | user | Одне (ownership) |
| PATCH | /api/orders/:id/status | admin | Змінити статус |
| DELETE | /api/orders/:id | admin | Видалити |

Тест створення замовлення

```
```text
```

```
<вивід curl POST /api/orders>
```

```
```
```

Тест ownership (403)

```
```text
```

```
<вивід curl GET /api/orders/:id з чужим токеном>
```

```
```
```

Тест зміни статусу

```
```text
```

```
<вивід curl PATCH /api/orders/:id/status>
```

```
```
```

Тест insufficient stock

```
```text
```

```
<вивід curl POST /api/orders з quantity > stock>
```

```
```
```

Troubleshooting — типові проблеми

1) "Cannot find module './entities/order.entity'"

Що зробити:

- Перевірити структуру папок: src/orders/entities/order.entity.ts
- Перевірити імпорти — шлях має бути відносним від файлу, який імпортує

2) Транзакція не відкочується — stock зменшується навіть при помилці

Що зробити:

- Перевірити що ви використовуєте `qr.manager.save()`, а НЕ `this.productRepo.save()`
- `this.productRepo` працює ПОЗА транзакцією — `rollback` його не скасує
- Всі операції з даними мають проходити через `qr.manager`

3) OrderItem зберігається без orderId

Що зробити:

- Перевірити що Order Entity має `cascade: true` на OneToMany до OrderItem
- При `cascade: true` достатньо зберегти Order з масивом items — TypeORM сам запише OrderItems з правильним orderId

4) "ILIKE" або фільтрація не працює в OrderQueryDto

Що зробити:

- Перевірити `@Type(() => Number)` на числових query параметрах
- Перевірити що `transform: true` увімкнено в ValidationPipe (main.ts)

5) Ownership check не працює — user бачить чужі замовлення

Що зробити:

- Перевірити що `WHERE userId` додається для не-admin ролі
- Перевірити що `@CurrentUser('sub')` повертає числовий id (не рядок)
- Перевірити JWT payload: на jwt.io вставити токен і побачити sub та role

6) decimal ціна порівнюється некоректно

Що зробити:

- TypeORM повертає decimal як рядок. Використовуйте `Number(product.price)` при арифметичних операціях
- `totalPrice = Number(product.price) * item.quantity` — не `product.price * item.quantity`